LAB ASSIGNMENT

NLP Foundations: Handwriting OCR Evaluation

Deepening your understanding after Lab 6

Course	Artificial Intelligence
Practical	Lab 6 of 8 - Handwriting OCR Evaluation
Student details	Name: Nur Natasya Dania Binti Yaromar Student ID: 24005411

Purpose

This assignment is to be completed after the Lab 6 practical. You do not need to repeat the entire experiment. Instead, use what you learned to explain the concepts, interpret the results, reason about OCR errors and suggest improvements. Answer in your own words.

Instructions

1. Complete the assignment individually.
2. Use your Lab 6 practical outputs and the results discussed during the lab as evidence.
3. Show all calculation steps. Round numerical answers to 2–3 decimal places where appropriate.
4. You may refer to your lab results; you do not need to rerun the complete OCR experiment.
5. Include screenshots or small tables only where requested. Ensure that all evidence is readable.

Part A - Understand the concepts

Q1. NLP pipeline

Describe the five stages of an NLP pipeline: acquire, normalise, represent, model and evaluate. For each stage, give an example of what happens when a handwritten medicine-name image is processed.

1. Acquire: Collect the raw data. A photo or scan of a handwritten prescription or medicine label is captured with a camera or scanner, then loaded as an image file. Example: a photo of the handwritten name “Azimax 40mg” is taken and stored as a JPG.

2. Normalise: Clean the data so it is consistent. For images, this means resizing, converting to greyscale, removing noise and adjusting contrast. After OCR, the output text is also normalised (lowercasing, removing punctuation). Example: the photo is converted to greyscale and thresholded so the ink stands out from the paper.

3. Represent: Convert the cleaned data into a form the model can work with. Example: the image is split into word or character regions and turned into pixel arrays or feature vectors. After OCR, the text becomes a string or tokens.

4. Model: Apply the algorithm that makes the prediction. Example: an OCR model reads the image and outputs the text "azimax 40 mg".

5. Evaluate: Measure how good the output is by comparing it with the reference answer. Example: the OCR output is compared with the true label "Tab Azimax 40mg" using exact match or character error rate to see how accurate it was.

Q2. How OCR works

Explain the difference between segmentation and character recognition. Give one example of a failure at each stage. Why is it useful to identify which stage caused an error?

Segmentation is the step that finds and separates the pieces of text in the image, such as lines, words or individual characters. **Character recognition** is the step that decides what each separated piece actually is (for example, that a shape is the letter "a").

Failure example in segmentation: In cursive handwriting, letters touch each other, so "mm" might be split in the wrong place, or two words might be merged into one region.

Failure example in character recognition: The segment is correct, but the model misreads it, such as reading a handwritten "l" as "1" or "a" as "o".

It is very useful to identify the stage because the fix is different for each. A segmentation error needs better preprocessing or a better text-detection method, while a recognition error needs better training data or a better classifier. Knowing the stage avoids wasting time fixing the wrong part and makes debugging faster.

Q3. Text normalisation

The reference is "Tab Azimax 40mg" and the OCR output is "tab azimax 40 mg". Apply the Lab 6 normalisation rule (lowercase and keep only letters and digits). Show both normalised strings. Why must the same rule be applied to both?

Rule: lowercase, then keep only letters and digits (remove spaces and symbols).

- **Reference:** "Tab Azimax 40mg" = **tabazimax40mg**
- **OCR output:** "tab azimax 40 mg" = **tabazimax40mg**

After normalisation, both strings are identical, so they count as a match.

We apply the same rule so that the comparison is fair. If only one string were normalised, harmless differences like capital letters and spacing would be counted as errors. Using the same rule means only real differences in content are measured, not formatting differences.

Q4. Normalisation trade-off

The Lab 6 normaliser removes spaces. Describe one situation where removing spaces could hide an important difference. Suggest how you would change the normalisation rule for that situation.

For an instance, removing spaces can hide meaningful differences when the spacing separates different pieces of information. For example, the dose “Tab 5 0mg” (misread or split) and “Tab 50mg” both become “tab50mg”. Also, “Azimax 40 mg” vs. “Azimax 4 0mg” would look identical even though the OCR clearly misread the dose layout. In a medical setting, a hidden dosing error could be dangerous.

Another example: two different drugs listed together, such as “Tab A 10” and “Tab A10”, could be treated as the same, though the first may be a separate item and number.

Suggested change to the rule: Instead of deleting spaces, collapse multiple spaces into a single space and still lowercase and remove punctuation. Alternatively, keep a space between letters and digits so the drug name and the dose stay separate tokens (e.g. “azimax 40 mg”). You could also compare names and doses separately, using the strict rule for the dose.

Part B — Work with OCR measurements

Q5. Levenshtein distance

Calculate the Levenshtein distance between “breeky” (reference) and “beeky” (prediction). Identify the edit operation(s). Then calculate the normalised edit distance by dividing the distance by the length of the longer string.

Reference: “breeky” (6 letters)
Prediction: “beeky” (5 letters)

Compare them position by position:

- breeky: b r e e k y
- beeky: b _ e e k y

The prediction is missing the “r”. Everything else lines up, so one operation fixes it.

Levenshtein distance = 1

Edit operation: one deletion (the “r” in the reference is missing from the prediction). Viewed the other way, turning “beeky” into “breeky” needs one insertion of “r”. Either way, the cost is 1.

Normalised edit distance = distance ÷ length of the longer string
= 1 ÷ 6 = **0.167** (about 16.7%)

The longer string is “breeky” with 6 characters.

Q6. Character Error Rate (CER)

An OCR system produces 4 substitutions, 2 deletions and 1 insertion. The reference contains 35 characters. Calculate CER using CER = (S + D + I) / N. Explain what the result means. Can CER be greater than 1? Explain briefly.

Given: S = 4, D = 2, I = 1, N = 35

CER = (S + D + I) / N
CER = (4 + 2 + 1) / 35
CER = 7 / 35
CER = 0.20 (20%)

The result shows that on average, 20% of the reference characters were wrong. Put another way, about 1 in every 5 characters needed an edit to match the reference. Lower is better, and 0 means a perfect transcription.

CER can be greater than 1. CER divides by the length of the *reference* only, but insertions add errors without adding to that length. If the OCR outputs a lot of extra characters (for example, 10 reference characters but 15 errors including many insertions), the total errors can exceed N, giving a CER above 1 (over 100%). So CER is not capped at 1.

Q7. Compare the metrics

Explain exact-match accuracy, mean normalised edit distance, CER and blank predictions. For each metric, state whether a higher or lower value is generally better. Why is exact-match accuracy alone insufficient?

Metric	What It Measures	Better Value
Exact-Match Accuracy	The percentage of predictions that match the reference perfectly (after normalization).	Higher
Mean Normalised Edit Distance	The average, across all samples, of edit distance divided by the longer string's length. Shows how far off predictions are on average.	Lower
CER	The total character-level errors (S + D + I) are divided by the total number of reference characters.	Lower
Blank Predictions	How many images were where the OCR returned nothing. These indicate complete failures to read.	Lower

Exact-match accuracy alone is insufficient because It does not show the type of failure. It cannot tell you whether errors come from blank outputs, small character slips, or major misreads. Other than that using exact-match accuracy alone is considered as it is all-or-nothing. A prediction with one wrong character counts as just as wrong as one that is completely garbage. "beeky" vs. "breeky" scores 0, the same as a totally unrelated output.

Part C — Interpret the Lab 6 experiment

Use the results from Lab 6. Both pipelines were evaluated on the same 89 labelled images, with the same normalisation rule and fixed Tesseract configuration.

Q8. Read the results

Describe at least two improvements shown by the preprocessed results. Use values from the table as evidence.

Matric	Baseline	Grayscale + Border + Resize	Change
Exact-Match Accuracy	0.0 (0%)	0.011 (1.1%)	Higher
Mean Normalized Edit Distance	0.869	0.785	Lower
Character Error Rate	0.883	0.807	Lower
Blank Predictions	42	15	Lower

Preprocessing (grayscale, border, resize) improved the OCR results in three ways. First, blank predictions dropped from 42 to 15, so the OCR detected text in many more images (e.g. "bisleri" went from blank to "Bus ld"). Second, the mean normalized edit distance fell from 0.869 to 0.785 and CER fell from 0.883 to 0.806, meaning predictions were closer to the truth. Third, exact-match accuracy rose from 0.0 to 0.011, with "ostium" read perfectly. However, the gains are small and CER remains high, so preprocessing alone is not enough for handwriting.

Q9. Explain the result pattern

CER and mean normalised edit distance improved, and blank predictions decreased. However, exact-match accuracy increased only from 0.000 to 0.011. Explain how preprocessing can help recover text without making the complete medicine name correct.

Exact match only counts a prediction as correct if **every character** is right. Preprocessing (grayscale, border, resize) mostly helps Tesseract *see and locate* the handwriting, but it does not teach the model to read cursive letters correctly. So it can recover part of the text without getting the whole name right.

Evidence from the results:

- **Blank - Partial text:** "bisleri" went from blank to "Bus ld", and "tab breaky" went from blank to "Bveaky". The text is now detected, but several letters are still wrong, so exact match stays at 0 for these.
- **Closer but not correct:** "dicloran" became "diclorew", with only 2 edits (distance 0.25). Almost right, but one wrong character still counts as a failure for exact match.
- **Metrics reward partial credit:** CER and edit distance count how many characters are wrong, so every partly recovered word lowers them. Exact match gives no credit for "almost".

So CER fell from 0.883 to 0.806 and blanks dropped from 42 to 15, while exact match rose only from 0.000 to 0.011 (just "ostium"). The remaining errors come from the OCR model's recognition limits on handwriting, which image cleaning cannot fix.

Q10. Controlled experiment

Why did the lab keep the Tesseract settings (--oem 1 --psm 7), the 89 images and the normalisation rule unchanged? What would be difficult to conclude if several settings changed at once?

The lab kept **--oem 1 --psm 7**, the **same 89 images** and the **same normalisation rule** so that the only thing that changed was the **preprocessing**. This makes it a fair comparison, where any difference in the metrics can be attributed to preprocessing.

- **Same Tesseract settings:** a different engine mode (oem) or page segmentation mode (psm) changes how Tesseract reads the image by itself.
- **Same 89 images:** a different image set could be easier or harder, changing the scores for reasons unrelated to preprocessing.
- **Same normalisation rule:** a different rule changes what counts as a match, which would change exact match and edit distances.

If several things changed at once, you could not tell which change caused the improvement (or the drop). For example, if CER improved after changing preprocessing *and* psm together, you would not know whether preprocessing, psm, or the combination was responsible. The results would not be reliable evidence for any single change.

Q11. Diagnose the failure

Identify the likely category for each case and explain:

(a) Tesseract raises an error while processing an image;
Category: OCR/tool failure (system or input error). The problem is not the handwriting but the pipeline: a corrupted or unreadable image file, an unsupported format, a bad image shape after preprocessing, or Tesseract not being installed or found correctly. Fix: check the file, the preprocessing code and the Tesseract setup.

(b) OCR returns an empty string;
Category: no text detected (detection/segmentation failure). Tesseract ran without an error but did not find any text it was confident about. Likely causes are low contrast, faint or messy handwriting, noise, or text that is too small or cut off by the border. This matches the no text detected label in your error_analysis.csv (e.g. “breeky”, “azitma”). Fix: better preprocessing such as contrast, thresholding and resizing.

(c) OCR returns “nidu” when the reference is “nims”.
Category: character confusion (recognition failure). Text was found and segmented, but some characters were misread: “m” became “d” and “s” became “u”, while “n” and “i” were correct. This is 2 substitutions, giving a normalised edit distance of 2/4 = 0.5, which matches your baseline row for “nims”. Fix: a better recogniser or model trained on handwriting, since the problem is not detecting the text.

Part D — Apply your understanding

Q12. Suggest improvements

Suggest TWO improvements you would investigate next to improve handwritten OCR. For each, explain which error it targets and how you would test whether it helps. Do not claim an improvement works without evidence.

Improvement 1: Adaptive thresholding (binarisation) and contrast enhancement (e.g. CLAHE)

- **Error it targets:** Detection failures (blank predictions). The results show 15 images still returned blank after preprocessing (e.g. “breeky”, “azitma”, “tonoflox”). The photos have uneven lighting, shadows and faint ink, so contrast should help Tesseract find the text.
- **How to test:** Add this as a new pipeline, e.g. grayscale_border_resize_threshold. Keep everything else the same (--oem 1 --psm 7, the same 89 images, the same normalisation rule). Then compare against the current pipeline using all four metrics, especially blank predictions (currently 15) and CER (currently 0.8065). If blanks and CER do not improve, or if the number of images that got worse is large, the change should not be claimed as helpful.

Improvement 2: A handwriting-specific OCR model instead of Tesseract (e.g. TrOCR, or fine-tuning on this dataset)

- **Error it targets:** Character confusion (recognition failure). Even when text is detected, outputs like “Bveaky” for “breeky” or “Nabe” for “nims” show the model cannot read cursive well. Tesseract is mainly designed for printed text, so preprocessing alone cannot fix this.
- **How to test:** Run the new model on the same 89 images and apply the same normalisation. Compare exact-match accuracy, mean normalised edit distance and CER against Tesseract’s results. Since the dataset is small, I would split it into training and test sets for fine-tuning, and only evaluate on images the model has not seen. Otherwise the result would be unfairly high.

Q13. Medical safety scenario

A clinic wants to use OCR to read medicine names and dosages from handwritten prescriptions. Explain at least THREE risks of using OCR output without human checking. Recommend a safer workflow.

THREE risks of using OCR output without human checking:

1. **Wrong drug name.** Your results show a CER of about 0.81 and only 1.1% exact match, so most names are misread. A name such as “Azimax” could be read as a different drug, causing the wrong medicine to be dispensed.
2. **Wrong dosage.** Digits are easily confused in handwriting (e.g. “40mg” read as “4 0mg”, “1” vs “l”, “5” vs “s”). A dose error of 10 times higher or lower could cause overdose or an ineffective treatment.
3. **Accountability.** If harm occurs, it is unclear who is responsible, since no person verified the data.

Recommended safer workflow:

1. Scan the prescription and run OCR as an assistant, not as the final decision-maker.
2. Match OCR output against a database of valid medicine names and standard doses and flag anything that does not match or has low confidence.
3. Any blank, low-confidence or flagged result goes to a pharmacist for manual review.
4. A trained pharmacist or clinician verifies **every** medicine name and dose against the original prescription before anything is dispensed.
5. Confirm unclear cases with the prescribing doctor.
6. Keep an audit log and regularly measure OCR error rates to monitor performance.

The key principle is that **OCR speeds up data entry, but a human makes the final safety decision.**

Q14. Reflection

What is the most important thing you learned from Lab 6? Identify one concept you initially found difficult and explain it using an example from the practical.

The most important thing I learned from Lab 6 is that one metric alone can give a misleading picture of a system, and that preprocessing can help without solving the whole problem. At first, I found the difference between exact-match accuracy and CER or edit distance difficult to understand. After preprocessing, exact match rose only from 0.000 to 0.011, so it looked like almost nothing had improved. However, CER fell from 0.883 to 0.806 and blank predictions fell from 42 to 15. For example, “tab breaky” went from a blank output to “Bveaky”, which is still wrong for exact match but much closer to the truth, with a normalized edit distance of 0.444 instead of 1.0. This showed me that exact match gives no credit for partial success, while edit distance and CER measure how close the output is. I also learned that a controlled experiment, changing only the preprocessing, is needed to be sure that the improvement came from that change.

Submission to your AI Lab Git repository

IMPORTANT: Submit your completed assignment to the Git repository you created for the AI Lab.

Do not keep the assignment only on your computer or send it only through a private message. Upload it to your existing AI Lab Git repository.

6. Create a folder named Lab06_Assignment.
7. Save your completed report as Lab06_OCR_Assignment_YourName.pdf (or .docx).
8. Put supporting screenshots or figures in a subfolder named evidence.
9. Commit and push your changes to your existing AI Lab repository(if not done yet).
10. Open the repository online and confirm that your files are visible.

Suggested repository structure

```
AI-Lab-Repository/
└── Lab06_Assignment/
    ├── Lab06_OCR_Assignment_YourName.pdf
    └── evidence/
        ├── example_1.png
        └── results_table.png
```

Assessment focus

Your work will be assessed on conceptual understanding, correct calculations, interpretation of the actual Lab 6 results, quality of reasoning, clarity of writing and completeness of the Git repository submission.

Final reminder

The purpose is understanding, not simply reproducing code or copying results. Explain what the results mean, why the experiment was designed this way and what you would investigate next.